

Graduate Algorithms

Tutorial 2: Sorting and Searching

August 17, 2026

Roughly in 2021, some bicycle thieves were prowling the streets of England and Wales. Tom Whipple, a science correspondent for Times, had seven of his cycles stolen. This was, understandably, irritating. However, the 8th cycle was stolen from right below a security camera. So he reports it to the police and they say... “we can’t¹ do anything unless you can pinpoint a 30 min segment of time when it was stolen. We can’t waste time watching the whole footage.”

Whipple found out that this was quite a common complaint, from his article:

¹read: won’t

I found a chatroom thread among Cambridge computer scientists, one of whom had also been told that unless he could pin down the moment of theft no one would look at the footage. He said he had tried to explain algorithms to police — he was a computer scientist, after all.

You don't watch the whole thing, he said. You use a binary search. You fast forward to halfway, see if the bike is there and, if it is, zoom to three quarters of the way through. But if it wasn't there at the halfway mark, you rewind to a quarter of the way through. It's very quick. In fact, he had pointed out, if the CCTV footage stretched back to the dawn of humanity it would probably have only taken an hour to find the moment of theft. This argument didn't go down well.

You can read the full article [here](#).

While the police was unable² to understand why this would be faster, could you try to explain why this would perhaps be faster?

²read: unwilling

The above idea is formally named **Binary Search**. Let's look at a few other applications.

Exercise: Peak

Given an array A , an element $A[i]$ is a peak if it is not smaller than its neighbor(s): $A[i] \geq A[i-1]$ and $A[i] \geq A[i+1]$ with imagining $A[-1] = A[n] = -\infty$. We want to find a peak.

A simple idea would be scanning the list and check the 'peak-iness' of each of the elements. This would have the worst case complexity of $O(n)$.

Another algorithm would be to just find the maximum element of the array.

However, consider the observation: **every rising sequence ends in a peak.**

“Well, that is still $O(n)!$ ” one might complain.

That is if we start from the head of the array. What if we check a middle element? If it is a peak, we are done. Otherwise, one of the sides is larger and we can just consider that half of the array.

Solution to Peaks

```
1. func solve(A, k):  
    a.  $k = \lfloor \text{len}(A)/2 \rfloor$   
    b. if  $A[k] \geq A[k-1]$  and  $A[k] \geq A[k+1]$ :  
        i. return k  
    c. else:  
        i. if  $A[k] < A[k-1]$ :  
            -- return solve(A[0, k-1])  
        ii. if  $A[k] < A[k+1]$ :  
            -- return solve(A[k+1, n-1])
```


This clearly takes $O(\log(n))$ time.

Exercise: Peaks 2D

Given a matrix M of numbers, we want to find a peak that is: an entry that is not smaller than its (up to) 4 neighbor.

Simply considering the center element doesn't work.

Finding the maximum of the whole matrix takes $O(n^2)$ which is perhaps 'too slow'.

We could sort of do something similar: find the maximum element in the middle column. If it is a peak, we are done. Else, we can look into the columns on the side with the greater elements.

This would take $O(n \log(n))$ time.

But could we do better?

Consider: Find the maximum element in middle row and column. If it is a peak, we are done. Else, look into the quadrant with greater element.

This reduced the $n \times n$ problem into a $n/2 \times n/2$ problem in some $O(n)$ time.

This gives us $T(n) = T(n/2) + O(n) \Rightarrow T(n) = O(n)$.

Exercise: Machine

A factory has n machines which can be used to make products. Your goal is to make a total of g products.

For each machine, you know the number of seconds (k_i for machine i) it needs to make a single product. The machines can work simultaneously, and you can freely decide their schedule. What is the shortest time needed to make g products?

So how do we even go about this? Given an amount of time t , we can return the number of products we could manufacture in that amount of time as:

$$f(t) = \sum_i \left\lfloor \frac{t}{k_i} \right\rfloor$$

However, this function doesn't have a clean inverse. So what do we do? Well, we search over a range of possible values for the minimal value that works. A simple range to search could be $[1, g \times \min_i(k_i)]$.

Solution to Machine

```
1. func items_in_time(K, t):  
    a. ans = 0  
    b. for k in K:  
        i. ans +=  $\lfloor t/k \rfloor$   
    c. return ans  
2.  
3. func bin_search(K, g, low, high):  
    a. if low == high:  
        i. return low  
    b. mid =  $\lfloor (low + high)/2 \rfloor$   
    c. if items_in_time(mid)  $\geq g$ :  
        i. bin_search(K, g, mid, high)  
    d. else:  
        i. bin_search(K, g, low, mid)  
4.  
5. func solve(K, g):  
    a. return bin_search(K, g, 1,  $g \times \min(K)$ )
```

This has time complexity $O(n \log(g \min(K)))$ as every step in the binary search calls the `items_in_time` function that takes $O(n)$ time.

Exercise: Square Roots

Exercise: Square Roots

Without calling the inbuilt `sqrt` function, can you implement a function `square_root(n, e)` that returns a such that $a^2 - e \leq n^2 \leq a^2 + e$.

While using a raw binary search is infeasible as it would not terminate as the exact answer is seldom rational. However, we can keep the tolerance in mind and simply work as follows:

Solution to Square Root

1. `func square_root_search(n, e, l, h):`
 - a. $a = (l + h)/2$
 - b. if $a^2 \geq n^2 - e$ and $a^2 \leq n^2 + e$:
 - i. return a
 - c. if $a^2 < n^2$:
 - i. return `square_root_search(n, e, a, h)`
 - d. if $a^2 > n^2$:
 - i. return `square_root_search(n, e, l, a)`
- 2.
3. `func square_root_search(n, e):`
 - a. if $n > 1$:
 - i. return `square_root_search(n, e, 1, n)`
 - b. if $n \leq 1$:
 - i. return `square_root_search(n, e, 0, 1)`

Our search space is at most from 1 to n with the tolerance aka e . Finally, it is taking $O(1)$ to query and $O(1)$ to exclude.

Thus, $O(\log((n-1)/e)) = O(\log(n/e))$.

Idea: When is Binary Search Applicable?

Idea

An problem is killable via Binary Search if

- There exists a way to (cheaply) query such that half of the search space could be eliminated

- We have an easy way to exclude the eliminated elements from the search space

- Sometimes if the reverse of the problem is easier to solve

Note: Often the second property is inherited from an order that can be defined on the search space. In the above, the order is $\text{false} < \dots < \text{false} < \text{true} < \text{true} < \dots < \text{true}$ with respect to is the time enough to produce g items.

Remark

Further notice that the time complexity of the algorithm is $\mathcal{O}(\log_2(S)(q(n) + e(n)))$ where S is the size of search space, $q(n)$ is the time per query and $e(n)$ is the time to exclude $n/2$ elements from the search space.

Please mind the use of $\mathcal{O}$ and not Θ . For Θ , we would perhaps need to consider $\mathcal{O}(q(n) + q(n/2) + \dots + q(1) + e(n) + e(n/2) + \dots + e(1))$ or solution to the recursion $T(n) = T(n/2) + e(n) + q(n)$.

Exercise: Carnivel (CEIO 2014)

Exercise: Carnivel (CEIO 2014)

Each of Peter's N friends (numbered from 1 to N) bought exactly one carnival costume in order to wear it at this year's carnival parties. There are C different kinds of costumes, numbered from 1 to C . Some of Peter's friends, however, might have bought the same kind of costume. Peter would like to know which of his friends bought the same costume. For this purpose, he organizes some parties, to each of which he invites some of his friends.

Peter knows that on the morning after each party he will not be able to recall which costumes he will have seen the night before, but only how many different kinds of costumes he will have seen at the party. Peter wonders if he can nevertheless choose the guests of each party such that he will know in the end, which of his friends had the same kind of costume. Help Peter!

A naive solution would be having parties with each possible pairs of 2 people to discover if they share a costume. This would require $\binom{n}{2} = O(n^2)$ parties.

We could make the above process a bit faster. Every time we discover someone with a costume we have not yet seen, we designate them as 'head' of people with that costume. Every time we need to check if a new person has the same costume as someone else, we can simply have parties with the heads. This reduces the number of parties to $< n \cdot c = O(nc)$ which in the worst case is still $O(n^2)$.

But wait, didn't we do a search in above part? Consider the following position: we have heads $h_1, h_2, \dots, h_i$ and are considering the person p_j to figure if they have the same costume as one of the heads or a new costume. We can have a party $h_1, \dots, h_i, p_j$ to check if it is a new costume. If we get i , we now need figure out which head they have the same costume as. Else, we can just declare they have a new costume and make them a head.

Consider the result if we have a party $h_1, h_2, \dots, h_k, p_j$? If p_j has a costume same as any of the heads, we would get k else $k + 1$. This query immediately tells us if the head p_j shares a costume with is in $h_1, \dots, h_k$ or $h_{k+1}, \dots, h_i$. Isn't this exactly what we want to binary search?

Writing the full pseudo-code is left for you. The number of parties is clearly $< n \cdot \log(c) = O(n \log(c))$ which in the worst case is bound by $O(n \log(n))$.

A common binary search question in (data science) interviews is as follows:

Exercise: Holes

You are given an array A from 1 to n , in ascending order. However, the array has some holes (ie, missing numbers). Can you find the smallest hole?

I think you can clearly see the $O(\log(n))$ binary search solution. However, it is a cardinal rule in interviews that *thou must pretend the question is hard before thou solves it*.

Holes: An Interview Dialogue

So our interviewee asked “How were the holes chosen?”

“Randomly”

“From what distribution?”

“Uniform”

“How many holes do we have?”

“Any number between 1 to n , randomly.”

“Uniformly?”

“Yup.”

“Hmm, In that case, I would just search sequentially.”

“Won’t that be terribly inefficient?”

“Not really. In expectation, it’s same time. But given how computer hardware works, I suppose it would actually be faster most of the time.”

“Hain?”

I suppose the interviewee had an uphill battle explaining computer hardware and probability to a person who probably wanted the answer matching the one from whatever website they were using.

Holes: The Mathematical Reasoning

The mathematical reasoning is, let $\mathcal{A}$ be the performance of the algorithm, h be the number of holes (which is uniformly distributed from 1 to n).

$$\begin{aligned}\mathbb{E}(\mathcal{A}) &= \mathbb{E}(\mathbb{E}(\mathcal{A} \mid h)) && \text{by Law of Iterated Expectations} \\ &= \mathbb{E}\left(\frac{n}{h+1}\right) && \text{by expectation of first order statistic} \\ &= \frac{\sum_{h=1}^n n/(h+1)}{n} && \text{by uniform distribution} \\ &= \frac{nH_n}{n} && \text{by definition} \\ &= H_n \\ &= O(\log(n))\end{aligned}$$

And on the hardware end, checking the first elements is usually marginally faster thanks to caching and hardware design.

Sadly, we could be even faster at the cost of more confusion.

We could begin by checking the last item of the list and that would tell us the number of holes. Now we check $A[n/h]$ and use the binary search idea here.

With a lot of work (search: interpolation search), we can prove the expected time complexity to be $O(\log(\log(n)))$.

This is also not as unintuitive as it seems. We do this naturally when searching for a book in a library shelf, instead of checking the half point for a book by an author with name 'J. Ericson', we check somewhere in the first fifth... around the 5/26-th point.

We have already seen the ideas of Quick and Merge sort in the class as well as have seen a proof that we can't sort in better than $O(n \log(n))$. We will analyze these both algorithms in some more detail next week.

However, say you have a bunch of spaghetti and you want to sort them. Well, just level them across the table. Then move your hand from top towards the tops of the spaghetti till your hand touches the tallest spaghetti, pull it out and continue. This is a $O(n)$ algorithm. It even has a name: Spaghetti Sort.

However, we just sorted spaghetti in $O(n)$ time, how?

Sorting: What is at Play?

The answer 'that was a stupid algorithm' is plain wrong as our proof for $O(n \log n)$ never relied on dumbness or smartness. So what is at play?

Similar to the interview story, knowing more about what we are sorting usually can help us sort faster. The $O(n \log n)$ proof was on a decision tree and made no assumptions about what we were sorting. So we could only use comparisons.

However, given that most sorting is of integers and strings, why would you ever so restrict yourself as to only use comparisons? You can do much more with these objects than that. You can add them, you can multiply them, you can count with them!

While it may seem unintuitive, but we can use even these operations for sorting!

Let's say we want to sort a range of $0-k$ with some number of repeats. So what do we do? We can do this in $O(n + k)$ time.

Counting Sort

```
1. func radix_sort(A, k):  
  a. count  $\leftarrow$  array of k 0s  
  b. for a in A:  
    i. count[a] += 1  
  c. ans  $\leftarrow$  empty array of length equal to A  
  d. ind = 0  
  e. for i in range(k):  
    i. for j in range(count[i]):  
      -- ans[ind] = i  
      -- ind += 1  
  f. return ans
```

For a fixed k , this is already an $O(n)$ algorithm; but it is easy to observe, that in practice, we would want to fix a large k , probably the maximum integer our system could store, around 2^{32} .

That would make the algorithm insanely slow while still being $O(n)$.
So why talk about it? Because it is a subroutine to...

We begin by first making modifications to our previous code to create `counting_sort_with_key` which will sort an array of pairs based on the key value of the pairs (the second value of the pair) keeping the list otherwise stable (we don't change the order from that prescribed by original list if key value is same).

Counting Sort with Key

```
1. func counting_sort_with_key(A, k):  
    a. count ← array of k 0s  
    b. for (a, b) in A:  
        i. count[b] += 1  
    c. ans ← empty array of length equal to A  
    d. ind = 0  
    e. for i in range(k):  
        i. for j in range(count[i]):  
            -- ans[ind] = (a, i)  
            -- ind += 1  
    f. return ans
```

Definition: Radix Sort

Definition: Radix Sort

Radix sort is a sorting algorithm which sorts a list of integers digit by digit, starting from the least significant digit; maintaining stability in subsequent sorts.

Radix Sort: Example

For example, if we had to sort:

853, 872, 265, 238, 199, 772, 584, 204, 480, 173, 499, 349, 308, 314, 317, 186, 825, 398, 899, 161

By the described process, we would first sort using the one's digit.

480, 161, 872, 772, 853, 173, 584, 204, 314, 265, 825, 186, 317, 238, 308, 398, 199, 499, 349, 899

We will now sort using the ten's digit, remember, we need to be stable that is for numbers that are tied on the middle digit, keep them in the current order.

204, 308, 314, 317, 825, 238, 349, 853, 161, 265, 872, 772, 173, 480, 584, 186, 398, 199, 499, 899

Finally, we will sort using the hundred's number.

161, 173, 186, 199, 204, 238, 265, 308, 314, 317, 349, 398, 480, 499, 584, 772, 825, 853, 872, 899

Exercise: Proof of Correctness

Exercise: Proof of Correctness

Show that Radix Sort correctly sorts an input list of n integers via induction on the length of longest number in the list.

Hint: You might want an induction hypothesis which looks more like the process. Something along the lines that the last j places are sorted in j th pass.

So how do we quickly sort the numbers by the last places? Use it as a key and use `counting_sort_with_key`.

This would have a time complexity of $O(n \log_{10}(M))$. We can do a bit better by choosing a relevant base. That would lead to a time complexity of $O(n \log_n(M)) = O(n \log(M) / \log(n))$.

In this survey, we will consider the optimal data structures. We might discuss some of these later.
We will let $w = \log(M)$

Published	Algorithm / Authors	Data Structure	Complexity
Since Antiquity	Merge Sort	List	$\mathcal{O}(n \log(n))$
Since Antiquity	Radix Sort	Array, List	$\mathcal{O}(nw / \log(n))$ and for list, $\mathcal{O}(nw)$
1974	van Emde Boas	van Emde Boas Tree	$\mathcal{O}(n \log(w / \log n))$
1983	Kirkpatrick, Reisch	Trie	$\mathcal{O}(n + w / \log(n))$
1995	Andersson, Hagerup, Nils-son, Raman (Signature Sort)	Compressed Trie	$\mathcal{O}(n \log \log n)$ for $\log^{2+\epsilon}(w) > n$
2002	Han, Thorup	Too Weird	$\mathcal{O}(n \sqrt{\log \log n})$ for some nice bound on w^3

³The reason we don't describe Han-Thorup Algorithm well as none of us are that very interested in sorting algorithms and hence, don't have the level of knowledge of tree structures and algorithms needed to do a description of this justice.

An open question is if it is possible to do sorting in $\mathcal{O}(n)$. For example, if $w = \Omega(\log(n)) \Rightarrow$ Radix Sort is $\mathcal{O}(n)$.

What about smaller w ? This was given by Andersson et al. where $\mathcal{O}(n)$ is achieved for $w = \mathcal{O}(n^{1/2-\epsilon})$.⁴

Belazzougui et al. in 2014 gave a way to sort in $\mathcal{O}(n)$ for $w = \Omega(\log^2(n) \log \log n)$. Their algorithm, called Packed Sort, works normally close to $\mathcal{O}(n \log n)$ but in certain cases becomes much faster.

A general proof or a single algorithm across w is not yet known and not much progress has been made in the last decade. Same is true for randomized algorithms like Quick Sort; while results are slightly better there, progress has slowed down considerably.

⁴One can also see the complexity when $\log^{2+\epsilon} w > n$ in the table. The middle cases are where a complex complexity (pun intended) form with w and n can be obtained.